

CNN IMAGE CLASSIFICATION

(FROM SCRATCH VS TRANSFER)

ThS. Lê Thị Thùy Trang

2026-04-12

1 Giới thiệu bài thực hành

1.1 Mục tiêu bài thực hành

Sau bài lab này, sinh viên sẽ chuyển từ mạng nơ-ron truyền thẳng (MLP) ở Lab 1 sang **mạng neuron tích chập (CNN)** cho bài toán phân loại ảnh lỗi bề mặt thép. Mục tiêu chính của buổi học không phải là chạy thật nhiều backbone cho oách, mà là hiểu vì sao CNN phù hợp với ảnh hơn MLP và biết cách triển khai một thí nghiệm bài bản với *Weights & Biases*.

- Hiểu vì sao CNN phù hợp với dữ liệu ảnh hơn MLP;
- Thực hành huấn luyện một mô hình CNN từ đầu;
- Làm quen với tư duy transfer learning: freeze và fine-tune;
- So sánh CNN from scratch với transfer learning trên cùng một tập dữ liệu;
- Đọc learning curves và phát hiện overfitting/ underfitting;
- Ghi lại toàn bộ quá trình bằng *Weights & Biases*;
- Rút ra kết luận dựa trên số liệu thay vì cảm giác.

1.2 Kết quả đầu ra mong đợi của bài lab

Sau bài lab này, sinh viên cần có:

1. Một repo chạy được với bài toán phân loại ảnh bằng CNN;
2. Một mô hình CNN from scratch chạy ổn trên tập `NEU-CLS.zip`;
3. Một mô hình transfer learning chạy được và có log trên *W&B*;
4. Ít nhất 2 run chính: scratch và transfer;
5. Learning curves cho train và validation;
6. Một bảng so sánh metrics và thời gian train/epoch;
7. Kết luận mô hình nào phù hợp hơn và vì sao.

1.3 Những thứ bắt buộc phải nộp

Mỗi sinh viên cần nộp tối thiểu:

1. Repo code chạy được.
2. README hướng dẫn chạy.
3. Link hoặc ảnh chụp W&B dashboard.
4. Kết quả của mô hình CNN from scratch.
5. Kết quả của mô hình transfer learning.
6. Bảng so sánh metrics + thời gian train/epoch.
7. Learning curves.
8. Kết quả test của best model.
9. Phân tích ngắn: khi nào transfer learning tốt hơn và khi nào chưa chắc cần dùng.

1.4 Bối cảnh bài toán

Chúng ta vẫn tiếp tục đóng vai nhóm kỹ sư AI hỗ trợ bộ phận QA trong dây chuyền sản xuất thép. Mỗi ảnh đầu vào là một ảnh grayscale chụp bề mặt thép. Mô hình cần dự đoán ảnh đó thuộc loại lỗi nào.

Bộ dữ liệu sử dụng vẫn là **NEU Surface Defect Database**, sinh viên truy cập đường dẫn [đây](#) để lấy dữ liệu. Tập dữ liệu gồm 6 lớp, tương ứng với các loại lỗi:

- Crazing
- Inclusion
- Patches
- Pitted Surface
- Rolled-in Scale
- Scratches

Nếu ở Lab 1 chúng ta phải **flatten** ảnh thành vector để đưa vào MLP, thì ở Lab 2 chúng ta để mô hình nhìn ảnh theo đúng cấu trúc không gian của nó. Đây chính là lúc CNN phát huy tác dụng.

1.5 Cấu trúc repo starter kit

Repo của bài lab có cấu trúc như sau:

```
csc4005_lab2_neu_cnn_starter/  
README.md  
REPORT_TEMPLATE.md  
requirements.txt  
configs/  
  baseline_scratch.json  
  baseline_transfer.json  
docs/  
  WANDB_GUIDE.md  
  LAB_GUIDE_LAB2.md  
notebooks/  
  lab2_demo.ipynb
```

```
outputs/  
src/  
  __init__.py  
  dataset.py  
  model.py  
  train.py  
  utils.py  
ci/  
  check_structure.py  
  smoke_train.py  
.github/  
  workflows/  
    validate-lab2.yml
```

Ý nghĩa từng phần

- `src/dataset.py`: đọc dữ liệu từ `NEU-CLS.zip` hoặc thư mục đã giải nén.
- `src/model.py`: định nghĩa CNN from scratch và các backbone transfer learning.
- `src/train.py`: huấn luyện mô hình, đánh giá, log W&B.
- `src/utils.py`: các hàm tiện ích như vẽ curves, confusion matrix, lưu metrics.
- `docs/WANDB_GUIDE.md`: hướng dẫn riêng cho W&B.
- `REPORT_TEMPLATE.md`: mẫu báo cáo cho sinh viên.

2 Giới thiệu công cụ Weights & Biases

Ở Lab 1, các em đã làm quen với W&B. Sang Lab 2, công cụ này vẫn là người bạn đồng hành cực kỳ quan trọng, vì bây giờ ta không chỉ so sánh optimizer hay dropout nữa, mà còn phải so sánh giữa **hai hướng tiếp cận hoàn toàn khác nhau**:

- CNN from scratch;
- Transfer learning.

Nếu chỉ nhìn log trong terminal, một lát là rất dễ rối:

- model nào train nhanh hơn;
- model nào hội tụ đẹp hơn;
- backbone nào val accuracy tốt hơn;
- run nào đang freeze hoàn toàn, run nào đang fine-tune;
- model nào tốn thời gian/epoch nhiều hơn.

W&B giúp mọi thứ được log lại rõ ràng, minh bạch và so sánh được.

W&B cho phép:

- Lưu cấu hình của từng lần chạy;
- Ghi lại train loss, val loss, train accuracy, val accuracy theo từng epoch;
- Theo dõi learning rate và thời gian train;
- So sánh nhiều run trên cùng một dashboard;

- Giữ lại best metrics để đưa vào báo cáo;
- Giúp thí nghiệm có thể tái lập được.

2.1 Lý do tiếp tục dùng W&B

Bài lab này yêu cầu sinh viên:

1. Huấn luyện 2 mô hình: scratch và transfer;
2. Có learning curves;
3. Có bảng tổng hợp metrics + thời gian train/epoch;
4. Kết luận khi nào transfer learning tốt hơn.

Nếu không dùng W&B, sinh viên vẫn có thể chạy mô hình, nhưng rất khó quản lý kết quả một cách khoa học. Với W&B, sinh viên không chỉ chạy được mà còn chứng minh được kết luận của mình bằng dữ liệu thí nghiệm. Tóm lại, W&B không làm mô hình thông minh hơn, nhưng làm quá trình thực nghiệm chuyên nghiệp hơn.

3 Chuẩn bị môi trường thực hành

3.1 Điều kiện cần

Sinh viên cần:

- Có tài khoản GitHub và đăng nhập được.
- Máy có Internet.
- Môi trường ảo csc4005-dl đã được set-up từ buổi học trước.
- Đã biết cách dùng wandb login từ Lab 1.

3.2 Clone repo

Bấm **Accept Assignment** theo link đã cung cấp trên Notion.

Clone repo về máy

```
git clone <repo-url>
cd <ten-repo>
```

3.3 Khởi tạo environment

```
conda activate csc4005-dl
```

3.4 Cài thư viện phục vụ bài lab

Thực tế thì mình cài từ bài lab 1 rồi, nhưng vẫn đề file requirements.txt trong repo rồi và hướng dẫn ở đây cho nó chắc chắn nhé.

```
pip install -r requirements.txt
```

Nếu phần transfer learning báo lỗi liên quan đến torchvision, hãy kiểm tra lại phiên bản torch và torchvision sao cho tương thích.

3.5 Nhắc lại thao tác với W&B

Nếu máy đã đăng nhập từ buổi trước thì có thể bỏ qua. Nếu chưa, thực hiện lại:

```
wandb login
```

Dán API key khi được yêu cầu.

4 Voila, hướng dẫn thực hành ở đây

4.1 Chạy baseline đầu tiên: CNN from scratch

Tinh thần của bài lab này là không nhảy ngay vào transfer learning cho sang môm, mà phải thử một mô hình CNN gọn gàng trước để hiểu chính xác CNN đang học cái gì và học khác MLP ra sao.

Ta bắt đầu bằng một baseline scratch ổn định:

```
python -m src.train \
--data_dir /duong_dan/NEU-CLS.zip \
--project csc4005-lab2-neu-cnn \
--run_name cnn_small_baseline \
--model_name cnn_small \
--train_mode scratch \
--optimizer adamw \
--lr 0.001 \
--weight_decay 0.0001 \
--dropout 0.3 \
--epochs 20 \
--batch_size 32 \
--img_size 64 \
--patience 5 \
--augment \
--use_wandb
```

Nhiều tham số quá nhi, chịu khó đọc giải thích tham số nha

- `--data_dir`: **bắt buộc**, là đường dẫn tới NEU-CLS.zip hoặc thư mục dữ liệu.
- `--project`: tên project trên W&B. Trong lab này dùng thống nhất là `csc4005-lab2-neu-cnn`.
- `--run_name`: tên run.
- `--model_name`: tên mô hình. Với baseline đầu tiên dùng `cnn_small`.
- `--train_mode`: `scratch`, `transfer`, hoặc `finetune`.
- `--optimizer`: chọn optimizer, ví dụ `adamw` hoặc `sgd`.
- `--scheduler`: có thể là `none` hoặc `plateau`.
- `--lr`: learning rate.
- `--weight_decay`: hệ số regularization L2.
- `--dropout`: hệ số dropout.

- `--epochs`: số vòng lặp huấn luyện.
- `--batch_size`: số mẫu trong một batch.
- `--img_size`: kích thước ảnh sau resize.
- `--patience`: số epoch chờ để early stopping.
- `--augment`: bật augmentation cho train set.
- `--use_wandb`: bật logging lên W&B.

Chạy scratch xong rồi thì nhớ:

1. Mở dashboard W&B.
2. Kiểm tra train loss và val loss.
3. Kiểm tra train accuracy và val accuracy.
4. Ghi chú lại thời gian train/epoch.
5. Giữ run này làm mốc so sánh với transfer learning.

4.2 Vì sao CNN hợp với ảnh hơn MLP?

Đây là phần quan trọng nhất của buổi học. Nếu chỉ chăm chăm chạy code mà không nắm được ý này thì hơi phí.

MLP nhìn ảnh như một dãy số dài sau khi flatten. Nghĩa là hai pixel đứng cạnh nhau trong ảnh khi đi vào MLP không còn được hiểu là đang nằm cạnh nhau nữa. CNN thì khác:

- CNN giữ lại cấu trúc không gian của ảnh;
- kernel trượt trên ảnh để học đặc trưng cục bộ;
- weight sharing giúp số tham số ít hơn MLP rất nhiều;
- pooling và chồng nhiều lớp conv giúp receptive field tăng dần.

Nói ngắn gọn: với ảnh, CNN không chỉ học **có gì** mà còn học **ở đâu**.

4.3 Nếu nhìn những biểu đồ như nhìn bức vách thì không sao hết, đọc hướng dẫn dưới đây là ổn áp ngay

Khi training xong, W&B sẽ cho chúng ta các đường cong học tập.

4.3.1 Những đường quan trọng nhất

- `train_loss`
- `val_loss`
- `train_acc`
- `val_acc`
- `epoch_time_sec`

4.3.2 Trường hợp tốt

- `train_loss` giảm đều,
- `val_loss` giảm đều hoặc giảm rồi ổn định,
- `train_acc` tăng,
- `val_acc` tăng theo.

Đây là dấu hiệu mô hình đang học đúng hướng.

4.3.3 Dấu hiệu overfitting

- `train_acc` tăng rất cao,
- `train_loss` tiếp tục giảm,
- nhưng `val_loss` bắt đầu tăng,
- `val_acc` không tăng tương ứng hoặc giảm.

Lúc này mô hình đang ghi nhớ train set quá mức.

4.3.4 Dấu hiệu underfitting

- cả train accuracy và val accuracy đều thấp,
- train loss và val loss đều còn cao.

Lúc này mô hình chưa học đủ hoặc mô hình quá yếu.

4.4 Đến phần được trông chờ đây: transfer learning

Bây giờ mới đến lúc dùng backbone pretrained. Ý tưởng là ta mượn một mô hình đã học rất nhiều đặc trưng ảnh trước đó, rồi điều chỉnh nó cho bài toán 6 lớp của mình.

Chạy baseline transfer như sau:

```
python -m src.train \
--data_dir /duong_dan/NEU-CLS.zip \
--project csc4005-lab2-neu-cnn \
--run_name resnet18_transfer \
--model_name resnet18 \
--train_mode transfer \
--optimizer adamw \
--lr 0.001 \
--weight_decay 0.0001 \
--dropout 0.3 \
--epochs 10 \
--batch_size 32 \
--img_size 128 \
--patience 3 \
--augment \
--use_wandb
```

Ở chế độ transfer, ta thường freeze phần backbone và chỉ train classifier head. Nếu muốn mạnh tay hơn, có thể thử finetune để mở thêm các lớp phía sau.

Ví dụ:

```
python -m src.train \
--data_dir /duong_dan/NEU-CLS.zip \
--project csc4005-lab2-neu-cnn \
--run_name resnet18_finetune \
--model_name resnet18 \
--train_mode finetune \
--optimizer adamw \
--lr 0.0001 \
--weight_decay 0.0001 \
--dropout 0.3 \
--epochs 10 \
--batch_size 32 \
--img_size 128 \
--patience 3 \
--augment \
--use_wandb
```

4.5 So sánh các run trên W&B

Sau khi chạy ít nhất 2 hướng tiếp cận, vào project trên W&B và so sánh.

- run nào có `best_val_acc` cao nhất,
- run nào có `val_loss` thấp nhất,
- run nào learning curve ổn định hơn,
- run nào train nhanh hơn theo epoch,
- run nào ít overfitting hơn.

Lưu ý: Kết luận phải dựa trên số liệu và dashboard, không nên viết cảm tính.

4.6 Bước cuối cùng: so bó đũa, chọn cột cờ

4.6.1 Chọn best model

Best model được chọn dựa trên kết quả train model.

Nguyên tắc đúng là:

1. So sánh các mô hình trên **validation set**.
2. Chọn **một cấu hình tốt nhất**.
3. Ghi rõ lý do chọn.
4. Chỉ sau đó mới dùng **test set** để đánh giá cuối cùng.

Tiêu chí chọn best config có thể là:

- val accuracy cao nhất,
- val loss thấp và ổn định,

- learning curves đẹp,
- ít overfitting hơn,
- thời gian train/epoch hợp lý hơn.

Hết sức lưu ý: Không được chọn mô hình chỉ vì train accuracy cao.

4.6.2 Đánh giá trên test set

Sau khi chọn được best config, sinh viên đánh giá trên test set **một lần cuối**.

Kết quả nên có:

- test accuracy,
- classification report,
- confusion matrix,
- một vài ví dụ dự đoán đúng,
- một vài ví dụ dự đoán sai.

Mỗi run sẽ được lưu trong:

```
outputs/<run_name>/
```

Thông thường sẽ có:

- best_model.pt
- history.csv
- curves.png
- confusion_matrix.png
- metrics.json

5 Khi nào transfer learning tốt hơn?

Đây là câu hỏi quan trọng của lab.

Transfer learning thường tốt hơn khi:

- dữ liệu không quá lớn;
- muốn hội tụ nhanh;
- muốn tận dụng đặc trưng ảnh đã học từ trước;
- cần một baseline mạnh trong thời gian ngắn.

Tuy nhiên, không phải lúc nào transfer cũng auto thắng. Nếu dữ liệu rất khác miền, hoặc backbone không phù hợp, hoặc fine-tune chưa đúng cách, kết quả có thể chưa hơn scratch bao nhiêu.

Bởi vậy, kết luận cuối cùng phải dựa trên số liệu của chính các em.

6 Những lỗi rất hay gặp

6.1 Lỗi 1. Freeze/unfreeze không đúng cách

Khi đó model có thể train rất chậm hoặc gần như không học được gì.

6.2 Lỗi 2. Resize sai chuẩn cho backbone pretrained

Một số backbone cần input lớn hơn và ổn định hơn. Không để ý chuyện này là kết quả đi xuống rất nhanh.

6.3 Lỗi 3. Không chuẩn hoá input phù hợp

Nhất là với pretrained model, input normalization rất quan trọng.

6.4 Lỗi 4. Chỉ nhìn accuracy mà không nhìn loss

Accuracy không phải lúc nào cũng kể hết câu chuyện. Learning curves rất quan trọng.

6.5 Lỗi 5. Không kiểm tra dữ liệu đầu vào

Sai đường dẫn `--data_dir`, sai tên file ảnh, hoặc dữ liệu không đúng định dạng sẽ làm training lỗi ngay từ đầu.

7 Câu hỏi tự kiểm tra sau lab

Hãy tự trả lời ngắn gọn các câu hỏi sau:

1. Vì sao weight sharing làm CNN hiệu quả hơn MLP cho ảnh?
2. Receptive field tăng lên như thế nào khi chồng nhiều lớp conv/pooling?
3. Khi nào nên chọn transfer learning thay vì train from scratch?
4. Vì sao cần so sánh scratch và transfer trên cùng một tập dữ liệu?
5. Dấu hiệu nào cho thấy mô hình bị overfitting?
6. W&B giúp ích gì khi phải so sánh nhiều cấu hình?

Nếu bạn trả lời rõ ràng được 6 câu này, nghĩa là bạn đã hiểu đúng tinh thần của lab.